

Vectorization of Linear Regression

Vectorization

- Benefits:
 - Compact representations
 - Faster implementations (using optimized matrix libraries)

- The model (for 1 observation): $h(x) = \sum_{j=0}^d \theta_j x_j$

- Let $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}$ $\mathbf{x}^T = [1 \quad x_1 \quad \cdots \quad x_d]$

- $h(x) = \boldsymbol{\theta}^T \mathbf{x}$

Vectorization

- Consider the model for n examples: $h(\underline{x^{(i)}}) = \sum_{j=0}^d \theta_j x_j^{(i)}$

- Let $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}_{(d+1) \times 1}$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_d^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix}_{n \times (d+1)}$$

$\leftarrow$ 1st training ex.

$\leftarrow$ i-th training ex.

The model becomes $h_{\theta}(x) = X\theta$

The Linear Regression Cost Function

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$= \frac{1}{2n} \sum_{i=1}^n (\vec{\theta}^T \vec{x}^{(i)} - y^{(i)})^2$$

$$\left[\begin{array}{l} \|z\|_2^2 \\ = z^T z \end{array} \right]$$

$$J(\theta) = \frac{1}{2n} \left[\underbrace{(\vec{X}\vec{\theta} - \vec{y})^T}_{1 \times n} \underbrace{(\vec{X}\vec{\theta} - \vec{y})}_{n \times 1} \right]$$

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \frac{1}{2n} [(\vec{X}\vec{\theta} - \vec{y})^T (\vec{X}\vec{\theta} - \vec{y})] \rightarrow 0$$

$$= \frac{1}{2n} \nabla_{\theta} \left[(\vec{X}\vec{\theta})^T \vec{X}\vec{\theta} - \underbrace{(\vec{X}\vec{\theta})^T \vec{y}} - \underbrace{\vec{y}^T (\vec{X}\vec{\theta})}_{\rightarrow 0} + \vec{y}^T \vec{y} \right]$$

Closed Form Solution

- Instead of using gradient descent, solve for optimal θ analytically

$$= \frac{1}{2n} \nabla_{\theta} \left[\theta^T (x^T x) \theta - y^T (x \theta) - y^T (x \theta) \right] \rightarrow 0$$

$$= \frac{1}{2n} \nabla_{\theta} \left[\theta^T (x^T x) \theta - 2 y^T (x \theta) \right] \rightarrow 0$$

$$= \frac{1}{2n} \nabla_{\theta} \left[\theta^T \underline{(x^T x)} \theta - 2 (x^T y)^T \theta \right] \rightarrow 0$$

$$= \frac{1}{2n} \left[2 x^T x \theta - 2 x^T y \right] \rightarrow 0$$
$$\hat{\theta} = \underline{(x^T x)^{-1}} x^T y$$

$$\begin{aligned} \nabla_x b^T x &= b \\ \nabla_x x^T A x &= 2 A x \end{aligned}$$

Gradient Descent vs Closed Form

Gradient Descent

- Requires iterations
- Need to choose α
- Works well when n is large
- Supports incremental learning

Closed Form Solution

- ✓ • Non-iterative
- ✓ • No need of α
- ✗ • Slower for larger n
- ✗ • The matrix $X^T X$ may not even be invertible

→ Pseudo-Inverse
(`numpy.linalg.pinv(a)`)

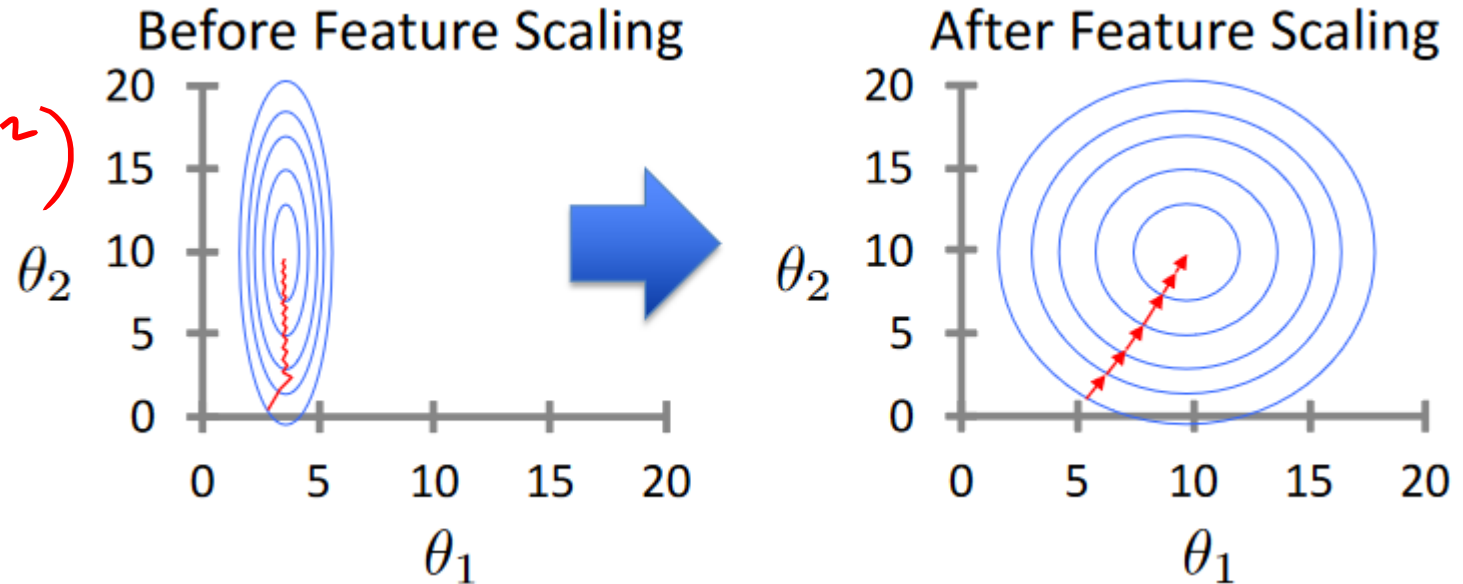
→ Remove redundant feature

→ $d \leq n$

Improving Learning by Feature scaling

- Feature Scaling

$x_1 = \text{Size (0 - 2000 ft}^2\text{)}$
 $x_2 = \# \text{ bedrooms (1 - 5)}$



- Makes Gradient Descent converge faster

$$\text{Let } h_{\theta}(x) = \theta_1 x_1 + \theta_2 x_2$$

$$x_1 \in \{0, 2000\}$$

$$x_1 \gg x_2$$

$$x_2 \in \{1, 5\}$$

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\frac{\partial J(\theta)}{\partial \theta_1} = \left[\frac{1}{n} \sum_i (h_{\theta} - y) x_1 \right] \quad \left. \begin{array}{l} x_1 \approx 1500 \\ x_2 \approx 3 \end{array} \right\}$$

$$\frac{\partial J(\theta)}{\partial \theta_2} = \left[\frac{1}{n} \sum_i (h_{\theta} - y) x_2 \right]$$

Small change in $\theta_1 \rightarrow$ large change in $\theta_1 x_1$
 Cost becomes extremely sensitive to θ_1

$$\frac{x_i - \mu_i}{\sigma_i} \leftarrow x_i$$

μ_i = mean of x_i

σ_i = range or standard deviation.

$$\frac{\partial J}{\partial \theta_1} \approx \frac{\partial J}{\partial \theta_2}$$

Contours will be more circular.

Feature scaling

- Rescales features to have zero mean and unit variance.
- Must apply the same scaling to both training and testing sets
- Outlier can be an issue